

WA3853 Lab Guide

CSS Fundamentals for Leaders



Part of
Accenture

© 2026 Ascendient, LLC

Revision 1.0.0 published on 2026-05-15

No part of this book may be reproduced or used in any form or by any electronic, mechanical, or other means, currently available or developed in the future, including photocopying, recording, digital scanning, or sharing, or in any information storage or retrieval system, without permission in writing from the publisher.

Trademark Notice: Product or corporate names may be trademarks or registered trademarks, and are used only for identification and explanation without intent to infringe.

To obtain authorization for any such activities (e.g., reprint rights, translation rights), to customize this book, or for other sales inquiries, please contact:

Ascendient, LLC

1 California Street Suite 2900

San Francisco, CA 94111

<https://www.ascendientlearning.com>

USA: 1-877-517-6540, email: getinfousa@ascendientlearning.com

Canada: 1-877-812-8887 toll free, email: getinfo@ascendientlearning.com

Lab Materials and Environment Guidelines

Ascendient Learning is pleased to provide lab materials and environments as part of your learning experience. To ensure a safe and effective learning experience for everyone:

● You may:

- Use the lab environment for your own coursework.
- Save any important files or data from the environment to your local computer or online storage (if allowed by your employer).
- Retain lab files for your personal use after the class ends.

● You may not:

- Share your lab materials or environment with anyone outside of your course.
- Store any personal, private, confidential, or proprietary company information within the lab environment.
- Rely on the lab environment for data storage, as all work is permanently deleted after each class.
- Misuse the lab environment for any illegal, malicious, or non-class-related activities.

🚨 Important

The lab environment is under Ascendient Learning's control and is not a part of your organization's IT ecosystem. Do not place your organization's information into the lab environment and always follow your organization's policies and procedures for working with IT systems.

Table of Contents

1. Evergreen Insurance Quote Page Brand Refresh with CSS Variables.....	5
1.1. A Note About Keyboard Shortcuts.....	6
1.2. Why You're Doing This Lab.....	6
1.3. What's Already Set Up on Your Lab VM.....	7
1.4. Open the Folder from the Command Line.....	7
1.5. Run the App in Chrome.....	8
1.6. Initialize Git for This Folder.....	9
1.7. Audit the Hardcoded Brand Color.....	10
1.8. Define CSS Variables at :root.....	11
1.9. Replace Hardcoded Colors with var(--...).....	12
1.10. Rebrand the Entire Page with One Variable Change.....	14
1.11. Inspect the Box Model in DevTools.....	16
1.12. Make the Padding Change Permanent.....	17
1.13. See the Whole Change in Git.....	17
1.14. Commit the Change.....	18
1.15. Review.....	19

Evergreen Insurance Quote Page Brand Refresh with CSS Variables

LAB 1

In this lab, you will open the Evergreen Insurance Quote Page in Visual Studio Code and refactor its stylesheet to use **CSS variables**. You will then change a single variable value and watch the entire site rebrand from Evergreen green to a new corporate blue. Along the way you will use Chrome DevTools to inspect the box model and adjust spacing.

You are not writing new visual design this week. You are moving **existing** colors and spacing values into a central place (CSS variables) so they can be changed in one line. This is exactly the work a design-system engineer does when the brand team requests a palette change.

1.1. A Note About Keyboard Shortcuts

Your lab VM runs inside your browser. Some keyboard shortcuts (like `Ctrl+Shift+X`, `Ctrl+S`, `Ctrl+F`) may not always reach the VM — sometimes your browser catches them first. If a shortcut doesn't do anything, that's not you doing something wrong; just use the mouse/menu alternative that appears next to every shortcut in this lab. Every step in this lab lists both.

1.2. Why You're Doing This Lab

CSS variables are one of the highest-leverage concepts in front-end work. A stylesheet with hardcoded colors is maintainable — until the brand team announces a repaint, at which point someone has to hunt every copy of `#0f5132` across dozens of files, update each one, and hope they got them all. A stylesheet with variables turns that multi-week project into a one-line change.

This lab shows you that exact transformation on a real (small) page. By the end you will have turned a stylesheet that repeats the brand green seven or eight times into a stylesheet that has the color in exactly one place — and then you'll change that one place from green to blue and watch the whole page update.

1.3. What's Already Set Up on Your Lab VM

Your lab machine runs Ubuntu Linux and comes pre-installed with everything you need for this lab. You do not need to install anything.

- **Visual Studio Code** — the code editor you used in Weeks 1 and 2.
- **Live Server extension** — already installed.
- **Git** — this week's lab folder will get its own fresh Git setup.
- **Firefox** and **Chrome** — either browser can be used. This lab asks you to use Chrome for the DevTools box-model diagram, because its panel is the clearest.

1.4. Open the Folder from the Command Line

You'll navigate to the lab folder using the terminal, then open it in VS Code from the command line.

1. Open a terminal. Three ways — use whichever feels easiest:
 - **From the dock:** Click the **Terminal** icon in the dock at the bottom of the screen (the black icon with `>_`).
 - **From Activities:** Click **Activities** in the top-left corner, type `Terminal` in the search box, then click the Terminal icon in the results.
 - **With a keyboard shortcut:** Press `Ctrl+Alt+T`.
2. Navigate into the lab folder:

```
cd ~/Labfiles/evergreen-quote-brand-refresh
```

Note

If you get "No such file or directory," run `ls ~/Labfiles` to see what's actually there.

3. Confirm you're in the right place:

```
ls
```

You should see `index.html`, `styles.css`, `app.js`, and `README.md`.

4. Open the folder in VS Code from the terminal:

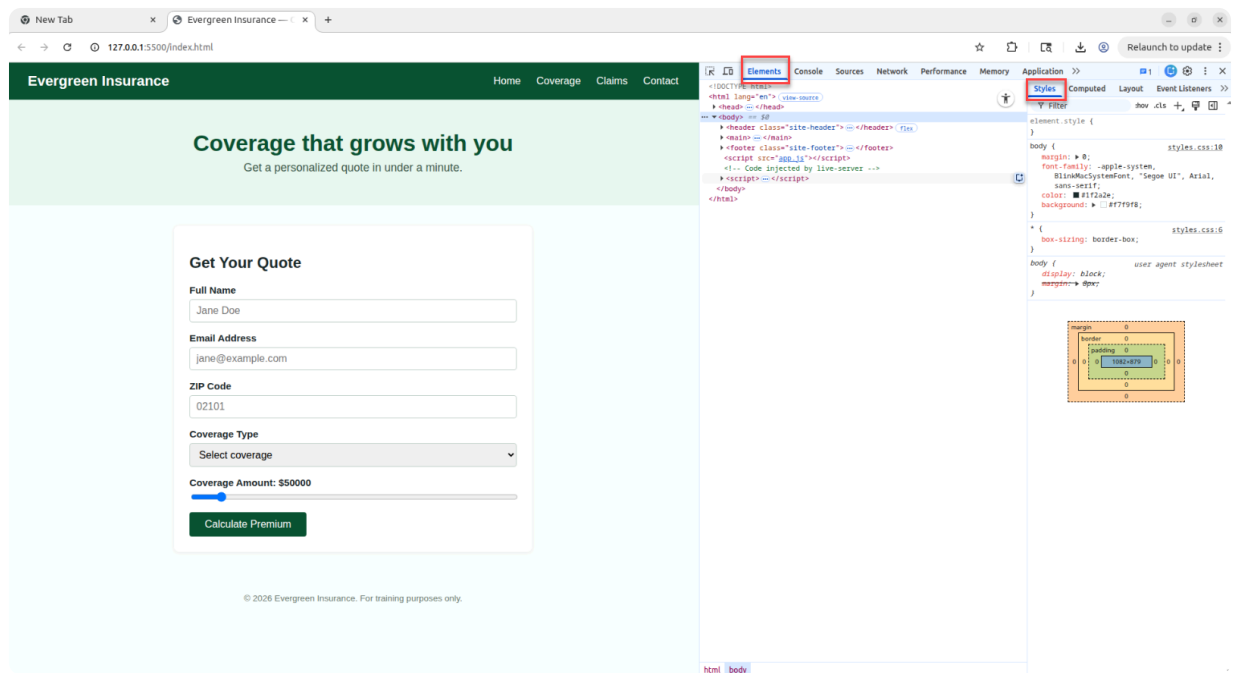
```
code .
```

Note

If VS Code asks "**Do you trust the authors of the files in this folder?**", click **Yes, I trust the authors**. The Evergreen files are provided by your course.

1.5. Run the App in Chrome

1. Open the Explorer panel if it is not already visible. Two ways:
 - **With the mouse:** Click the top icon in the Activity Bar (two overlapping pages).
 - **With a keyboard shortcut:** Press `Ctrl+Shift+E`.
2. Right-click on `index.html` in the Explorer and choose **Open with Live Server**.
3. The Evergreen quote page should open in your browser. If it opens in Firefox, right-click the Chrome icon in the dock and open the same URL in Chrome — this lab uses Chrome's DevTools box-model diagram.
4. The page should look the same as it did in Week 3: dark-green header, "Coverage that grows with you" hero, and a quote form including the email field you added last week.
5. Open Chrome DevTools. Two ways:
 - **With the mouse:** Click the three-dot menu in the top-right of Chrome, then **More Tools > Developer Tools**.
 - **With a keyboard shortcut:** Press `F12`.
6. Make sure the **Elements** tab is selected at the top of DevTools. You will use its **Styles** panel (on the right) throughout the lab. Click the **Styles** tab if it's not already active.
7. Leave the browser with DevTools open. Switch back to VS Code.



1.6. Initialize Git for This Folder

Just like in previous weeks, you need to tell Git to start tracking this folder before you can see changes.

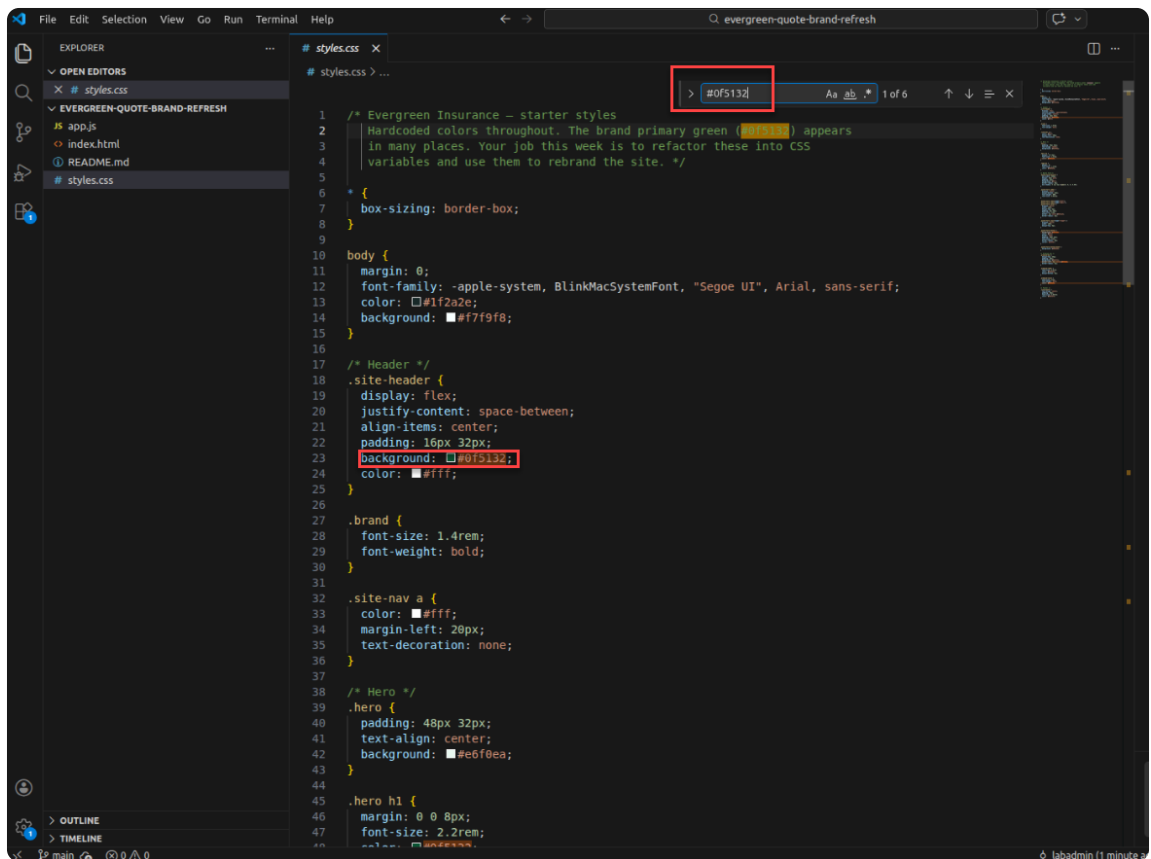
1. Open the Source Control panel. Two ways:
 - **With the mouse:** Click the third icon down in the Activity Bar (a branching line with dots).
 - **With a keyboard shortcut:** Press `Ctrl+Shift+G`.
2. Click the **Initialize Repository** button.
3. Make an initial commit so there is a baseline to compare future changes against:
 - Click inside the **Message** box at the top of the Source Control panel.
 - Type `week 4 starting state`.
 - Click the blue **Commit** button. If VS Code asks "Would you like to stage all your changes and commit them directly?", click **Yes**.
4. The Source Control panel should now be empty under **Changes**.

Why this step: A fresh baseline commit lets you view the entire Week 4 change as a single diff at the end of the lab.

1.7. Audit the Hardcoded Brand Color

Before you refactor anything, see the problem with your own eyes.

1. In the Explorer panel, click `styles.css` to open it in the editor.
2. Open the Find box. Two ways:
 - **From the menu:** Click **Edit > Find**.
 - **With a keyboard shortcut:** Press `Ctrl+F`.
3. In the Find box, type `#0f5132` (the Evergreen brand green) and press **Enter**.
4. Use the up/down arrow buttons next to the Find box to walk through each match. Count them as you go — VS Code will show **6 matches**. One is inside a comment at the top of the file; the other five are real style uses:
 - `.site-header` background (line ~23)
 - `.hero h1` text color (line ~48)
 - `.quote-form button` background (line ~93)
 - `.premium-box` border-left color (line ~111)
 - `.premium-value` text color (line ~124)
5. Now search for `#0b3d26` — the **darker** brand green used for the button's hover state. You should find one match, inside `.quote-form button:hover`. This is a separate shade, so it needs its own variable.



Why this matters: Seven copies of the same color value is small for one file; in a real multi-file stylesheet the count would be in the hundreds. If the brand team says "we're moving from green to blue," someone has to find and update each one. You are about to make that future change a one-line edit.

6. Close the Find panel with the `X` button or by pressing `Esc`.

1.8. Define CSS Variables at `:root`

CSS variables — officially called **custom properties** — are declared inside a rule and used anywhere a value would go. The convention is to declare them on the `:root` selector at the top of the stylesheet so they apply to the whole page.

1. At the very top of `styles.css` (just under the header comment), add a `:root` block:

```
:root {  
  --primary: #0f5132;  
  --primary-dark: #0b3d26;  
  --primary-tint: #e6f0ea;  
  --premium-bg: #eef7f1;  
  --text: #1f2a2e;  
  --text-muted: #3d5a4a;  
  --text-faint: #6a7a72;  
  --surface: #ffffff;  
  --page-bg: #f7f9f8;  
  --input-border: #bfc4c4;  
  --form-padding: 24px;  
}
```

2. Save the file. Two ways:

- **From the menu:** Click **File > Save**.
- **With a keyboard shortcut:** Press `Ctrl+S`.

3. Switch to the browser. The page should look **exactly the same** as before. You have not changed any visible value — you have only **defined** variables.

Why this matters: You introduced indirection without changing behavior. This is the safest possible refactor: add the new mechanism first, prove it doesn't break anything, **then** migrate the call sites over to use it. Every experienced engineer works this way.

4. Switch back to VS Code. You will use the variables you just defined in the next step.

1.9. Replace Hardcoded Colors with `var( - -... )`

Now you connect each hardcoded color to its new variable home.

1. In `styles.css`, find the `.site-header` rule. Change the `background` line:

```
.site-header {  
  ...  
  background: var(--primary); /* was #0f5132 */  
  ...  
}
```

2. Save. Switch to the browser. The header is still green — the variable resolved to `#0f5132`, and nothing visible changed.
3. Repeat for every other occurrence. In each rule, replace the hardcoded color with the matching variable:
 - `.hero h1` — `color: var(--primary);`
 - `.quote-form button` — `background: var(--primary);`
 - `.quote-form button:hover` — `background: var(--primary-dark);`
 - `.premium-box` — `border-left: 4px solid var(--primary);`
 - `.premium-value` — `color: var(--primary);`
4. While you're in the file, migrate the other shared values too:
 - `.hero` — `background: var(--primary-tint);`
 - `.premium-box` — `background: var(--premium-bg);`
 - `body` — `color: var(--text);` and `background: var(--page-bg);`
 - `.tagline`, `.premium-label` — `color: var(--text-muted);`
 - `.site-footer` — `color: var(--text-faint);`
 - `.quote-form-section` — `background: var(--surface);` and `padding: var(--form-padding);`
 - `.quote-form input[...]` — `border: 1px solid var(--input-border);`
5. Save. Switch to the browser. The page should still look exactly like before. If anything is suddenly the wrong color, use Ctrl+F in `styles.css` to look for leftover hex codes — you may have missed one.

Why this matters: The page hasn't visibly changed, but its source is now in a much better shape. Every color lives in exactly one place. Every engineer downstream who edits this file will thank you — and in a few minutes, so will you.

1.10. Rebrand the Entire Page with One Variable Change

Now for the payoff.

1. At the top of `styles.css`, change the values of `--primary` and `--primary-dark`:

```
:root {  
  --primary: #0066b2;          /* new corporate blue */  
  --primary-dark: #00497f;     /* darker companion */  
  ...  
}
```

2. Save.
3. Switch to the browser. The entire page should rebrand:
 - The header turns blue.
 - The hero heading "Coverage that grows with you" turns blue.
 - The **Calculate Premium** button is now blue, with a darker blue hover.
 - The premium box's left border and value text are both now blue.

The screenshot shows the Evergreen Insurance website. The header is blue with the logo 'Evergreen Insurance' and navigation links: Home, Coverage, Claims, Contact. Below the header is a green band with the text 'Coverage that grows with you' and 'Get a personalized quote in under a minute.' The main content area is light blue and contains a white box titled 'Get Your Quote'. Inside this box are input fields for 'Full Name' (Jane Doe), 'Email Address' (jane@example.com), and 'ZIP Code' (02101). There is a 'Coverage Type' dropdown menu with 'Select coverage' and a 'Coverage Amount' slider set to '\$50000'. A blue 'Calculate Premium' button is at the bottom of the form. At the very bottom of the page, there is a small copyright notice: '© 2026 Evergreen Insurance. For training purposes only.'

Why this matters: You just rebranded the site in one edit. Production design-system repos work exactly this way — a small `tokens.css` file defines the brand, and every component stylesheet uses variables declared there. When the brand team ships a refresh, the design-system engineer edits the tokens file, runs the test suite, and opens a PR. That PR, in principle, can change a thousand UI elements with a handful of lines.

4. Tweak the tint and premium box backgrounds to blue-friendly shades too:

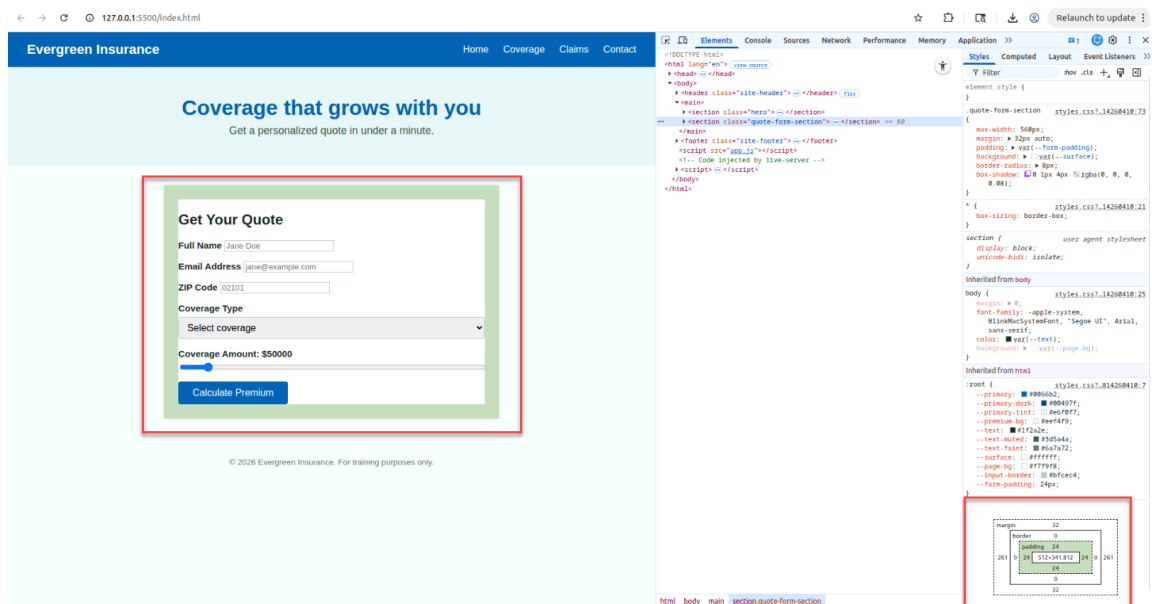
```
--primary-tint: #e6f0f7;  
--premium-bg: #eef4f9;
```

5. Save and watch the hero band and premium box backgrounds shift from green-ish to blue-ish. The page now feels consistently on-brand.

1.11. Inspect the Box Model in DevTools

You have one more CSS skill to exercise: reading the box model.

1. Switch to Chrome with DevTools open.
2. In the **Elements** panel, click on the `.quote-form-section` — the white card that holds the form. You can find it two ways:
 - **With the mouse:** Click the **Select Element** icon (the arrow in a box, top-left of DevTools), then click the white form card on the page.
 - **With a keyboard shortcut:** Press `Ctrl+Shift+C`, then click the form card.
3. In the right-hand **Styles** panel, scroll to the bottom until you see the colored **box model diagram**: four nested rectangles labeled *margin* (orange), *border* (yellow), *padding* (green), and the content box in the center.
4. Hover each rectangle. The browser highlights that region on the page in real time.



5. Double-click the **padding** value (currently `24`) in the DevTools box model diagram. Type `48` and press `Enter`. Watch the form card visibly grow — more whitespace around every input.

6. This change is **only in DevTools** — it will disappear when you reload the page. That's fine: DevTools is for experimenting. You just wanted to see what a padding change would look like before making it permanent.

1.12. Make the Padding Change Permanent

Now put that padding change into your actual stylesheet.

1. Switch back to VS Code with `styles.css` open.
2. At the top of the file, change `--form-padding` from `24px` to `32px`:

```
--form-padding: 32px; /* was 24px – more breathing room */
```

3. Save. Switch to the browser. The form card should now have noticeably more space around its inputs — not as dramatic as the `48px` DevTools experiment, but visibly roomier.

Why this matters: You made a real spacing change through a **variable**. If your team ever decides "every form on the site should have more breathing room," updating `--form-padding` in one place applies the change site-wide.

1.13. See the Whole Change in Git

1. Open the Source Control panel. Two ways:
 - **With the mouse:** Click the third icon down in the Activity Bar.
 - **With a keyboard shortcut:** Press `Ctrl+Shift+G`.
2. `styles.css` should appear under **Changes** with an `M`.
3. Click `styles.css` to open the diff view. Before and after side by side — you'll see the new `:root` block on the right, and the hardcoded colors replaced with `var(--...)` calls throughout.
4. Close the diff tab.
5. Open VS Code's integrated terminal. Two ways:
 - **From the menu:** Click **Terminal > New Terminal**.
 - **With a keyboard shortcut:** Press `Ctrl+``.

6. Run `git diff --stat`:

```
git diff --stat
```

You should see one file changed — `styles.css` — with a modest line count of insertions and deletions.

Why this matters: Even though the visual change is dramatic (whole-site rebrand), the diff is small and focused. That's the hallmark of a good refactor: big outcome, small surface area.

1.14. Commit the Change

1. In the Source Control panel, click inside the **Message** box and type:

```
Refactor styles to use CSS variables; rebrand to corporate blue
```

2. Click the blue **Commit** button. If VS Code asks to stage all your changes, click **Yes**.
3. Run `git status` in the terminal — it should say
`nothing to commit, working tree clean`.
4. Run `git log --oneline`:

```
git log --oneline
```

You should see your two commits — **Week 4 starting state** and **Refactor styles to use CSS variables; rebrand to corporate blue**.

Why this matters: A commit message like "Refactor styles to use CSS variables; rebrand to corporate blue" tells a future teammate exactly what this commit did and why. "Updated colors" would have told them nothing. Good commit messages add up to a changelog your team can actually use.

1.15. Review

In this lab, you:

- Opened the Evergreen stylesheet and counted hardcoded brand-color duplication.
- Added a `:root` block with CSS variables for the brand palette and common values.
- Replaced every hardcoded color in the stylesheet with a `var(--...)` reference, without changing anything visible.
- Changed two variable values and rebranded the entire site from Evergreen green to a new corporate blue.
- Used Chrome DevTools to read and experiment with the box model of the form card.
- Bumped `--form-padding` to tighten up the form visually.
- Reviewed the full change as a Git diff and committed it with a clear message.

You wrote no new JavaScript, added no new HTML elements, and changed roughly fifteen lines of CSS — but you rebranded a website. That is the leverage CSS variables give a design system.

Next week you'll revisit this same page in the Flexbox / Responsive Design module to make the header layout adapt to small screens.